

Proceed: Performance Routing Optimization for Cost Efficient and Effective Deployment

Vaibhav Gupta (2024201044), Sayam Agarwal (2024201025), Priyanshu Jha (2024201062)

Abstract

We present a multi-LLM routing system that directs queries to one of three language models **Mistral-7B**, **Mixtral-8x7B**, or **Zero-One Yi-34B** based on prompt complexity, optimizing for response quality, computational cost, and inference speed. Using a modified RouterBench dataset, we compute BERT similarity scores, costs, and throughput to generate composite scores, which are used to create soft labels for training a **roberta-large** routing model. The system achieves a **19.4%** cost saving compared to a baseline of always using Mixtral-8x7B, with a routing accuracy of **41.11%**. Our approach extends PolyRouter by incorporating a composite score for soft label generation, balancing multiple metrics, and focusing on larger LLMs for complex queries. Despite challenges such as dataset availability, LLM accessibility, and data bias, our system demonstrates practical utility, with avenues for further optimization.

1 Introduction

Large Language Models (LLMs) exhibit varying strengths depending on the complexity of the query, computational cost, and inference speed. Routing queries to the most suitable LLM can optimize performance while minimizing costs, a challenge addressed by recent benchmarks like RouterBench [1] and systems like PolyRouter [2]. In this project, we develop a routing system to direct queries to one of three LLMs—Mistral-7B, Zero-One Yi-34B and Mixtral-8x7B—selected to handle easy, moderate, and complex queries, respectively, balancing response quality, cost, and speed.

Our approach involves computing BERT similarity scores, costs, and throughput for each LLM, deriving composite scores to generate soft labels, and training a **roberta-large** model to predict the best LLM for a given prompt. The system is evaluated on a modified RouterBench dataset with 8,734 prompts, achieving significant cost savings compared to a baseline of always using Mixtral-8x7B. During inference, we deploy the system using the Together API, routing queries to the selected LLM and returning the response along with routing probabilities. However, a limitation arose as the Together API does not support Zero-One Yi-34B, requiring a substitution with another model, which introduces a potential mismatch between training and deployment.

Our work builds on the foundation laid by PolyRouter [2], a multi-LLM routing system that routes queries to a pool of expert LLMs, achieving up to 30% cost savings, 40% throughput improvement, and 10% performance enhancement. While PolyRouter uses BERT similarity scores alone to generate soft labels for training its BERT-Router, our system introduces a novel composite score approach, incorporating BERT similarity, cost, and throughput, to create more balanced soft labels. Additionally, we focus on larger LLMs (e.g., Mixtral-8x7B with 56 billion effective parameters, Zero-One Yi-34B with 34 billion parameters) to handle complex queries, compared to PolyRouter’s smaller models (up to 7 billion parameters). Our use of the RouterBench dataset, specifically designed for multi-LLM routing, further aligns our evaluation with routing-specific benchmarks, unlike PolyRouter’s use of multiple datasets (e.g., Ai2-ARC, GSM8k). This report details the methodology, results, and deployment of our routing system, highlighting its contributions, limitations, and comparisons with PolyRouter.

2 Methodology

2.1 Dataset

We use a modified version of the RouterBench dataset [1], originally designed for evaluating multi-LLM routing systems. RouterBench includes prompts across various domains, such as commonsense reasoning (Ai2-ARC), math (GSM8k), coding (HumanEval), and conversation (ChatBot Arena). After filtering to balance the dataset across our three target LLMs—Mistral-7B, Mixtral-8x7B, and Zero-One Yi-34B—we obtain 8,734 prompts. The dataset is split into 90% training (**7,860 prompts**) and 10% validation (874 prompts), with a separate test set of 1,745 prompts (approximately 20%) used for cost evaluation. The dataset is balanced to ensure roughly equal representation of each LLM, mitigating bias in routing decisions, as discussed in Section 5.

2.2 Composite Scores and Soft Labels

To train the routing model, we generate soft labels based on a composite score that integrates three metrics for each LLM: BERT similarity (quality), computational cost, and throughput (speed). The process is as follows:

1. **BERT Similarity Scores:** For each prompt, we compute BERT similarity scores between the ground truth response and the responses generated by each LLM, using a pre-trained BERT model. This measures the semantic alignment of the LLM’s response with the expected output.

2. **Cost and Throughput:** We calculate the inference cost (in USD per million tokens) and throughput (tokens per second) for each LLM based on API pricing and performance benchmarks. The costs are: Mistral-7B at \$0.2 per million tokens, Mixtral-8x7B at \$0.6 per million tokens, and Zero-One Yi-34B at \$0.8 per million tokens. Throughput values are measured during inference, as shown in Section 3.

3. **Composite Score:** For each prompt-LLM pair, we compute a composite score as a weighted sum of the normalized metrics:

$$\text{score} = w_1 \cdot \text{BERTSim}_{\text{norm}} - w_2 \cdot \text{cost}_{\text{norm}} + w_3 \cdot \text{throughput}_{\text{norm}}$$

where $w_1 = w_2 = w_3 = 1.0$, and each metric is normalized to $[0, 1]$ across the LLMs for that prompt. Normalizing cost as $1 - \text{cost}_{\text{norm}}$ ensures that lower costs contribute to a higher score.

4. **Soft Labels:** The composite scores for a prompt are converted into a probability distribution over the three LLMs using a softmax function with temperature $T = 0.5$:

$$p_i = \frac{\exp(\text{score}_i/T)}{\sum_{j=1}^3 \exp(\text{score}_j/T)}$$

where p_i is the probability for the i -th LLM. These probabilities serve as soft labels for training. We also define `soft_label_target` as the index of the LLM with the highest probability, used for evaluation and cost-saving calculations.

This approach ensures that the routing model learns to balance quality, cost, and speed, unlike PolyRouter’s BERT similarity-only soft labels.

2.3 Routing Model

We fine-tune a `roberta-large` model (355M parameters) for the classification task of selecting one of the three LLMs. The model is trained on the 7,860 training prompts with soft labels, using the Hugging Face `Trainer` API. The training setup includes:

- Learning rate: $2e-5$ - Batch size: 8 (training), 16 (evaluation) - Epochs: 15 (though results are reported for 5 epochs, see Section 3.1) - Optimizer: AdamW with weight decay 0.01 - Loss: Cross-entropy between predicted probabilities and soft labels

The model is evaluated on the validation set (874 prompts) using accuracy, defined as the proportion of prompts where the predicted LLM matches the `soft_label_target`. The best model is selected based on validation accuracy.

2.4 Inference and Deployment

After training, the routing model is deployed to handle real-time query routing using the Together API. The deployment process involves loading the trained `roberta-large` model and tokenizer, processing the input prompt to predict the most suitable LLM, and routing the query to the selected LLM via an API call. The system returns the selected model, the generated response, and the routing probabilities, allowing users to understand the router’s decision-making process.

A challenge arose during deployment: the Together API does not support Zero-One Yi-34B, one of the three LLMs used in training and evaluation. To address this, Zero-One Yi-34B was substituted with another model in the API calls. However, this substitution introduces a mismatch, as the substitute model may have different cost, throughput, and response quality characteristics compared to Zero-One Yi-34B, potentially affecting the reliability of routing decisions for prompts assigned to this LLM slot. This limitation is further discussed in Section 4. Future improvements could involve using an API that supports all target LLMs, as discussed in Section 6.

2.5 Model Workflow

To illustrate the overall workflow of our routing system, we provide a flowchart depicting the key steps from prompt input to response generation. The process encompasses both the training phase (computing metrics, generating soft labels, and training the model) and the inference phase (predicting the best LLM and routing the query).

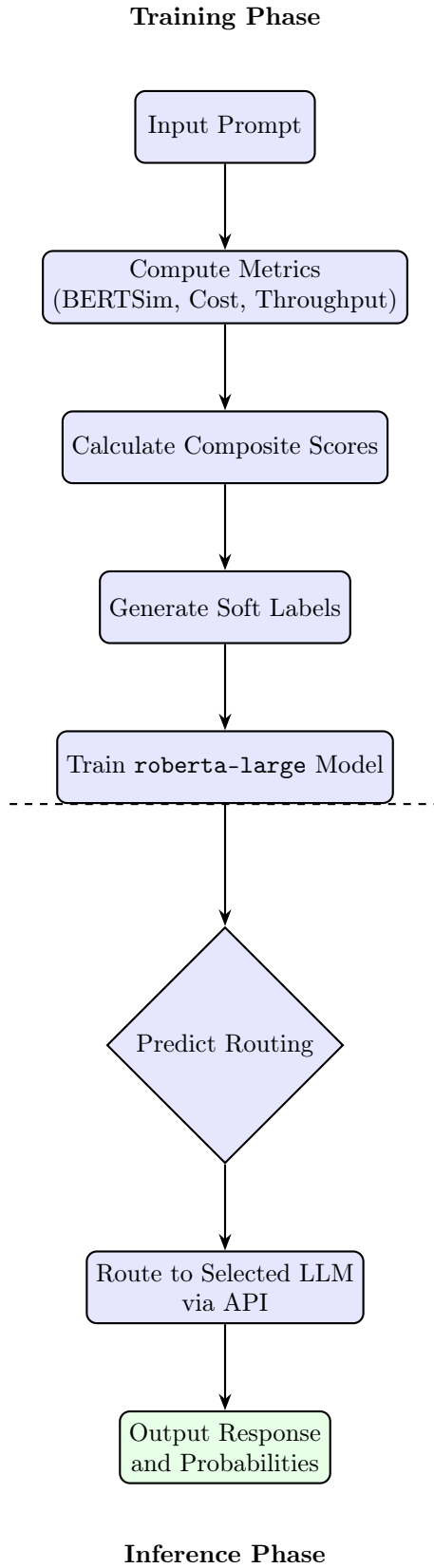


Figure 1: Flowchart of the multi-LLM routing system workflow, illustrating the training phase (computing metrics, generating soft labels, and training the **roberta-large** model) and the inference phase (predicting the best LLM and routing the query via API).

The flowchart (Figure 1) outlines the pipeline of our system. In the training phase, we start with an input prompt, compute the metrics (BERT similarity, cost, and throughput) for each LLM, calculate composite scores, generate soft labels, and train the **roberta-large** model. In the inference phase, the trained model predicts the best LLM for a new prompt, routes the query to the selected LLM via the Together API, and outputs the response along with routing probabilities. This visual representation highlights the end-to-end process of our routing system, emphasizing the distinction between training and inference.

3 Results

We evaluate the performance and cost efficiency of our routing system using the modified Router-Bench dataset, focusing on three key aspects: the individual metrics (BERT similarity, cost, and throughput) for the LLMs, the accuracy of the trained routing model, and the cost savings achieved by the router compared to baseline strategies.

3.1 Routing Model Performance

We evaluate the performance of the trained **roberta-large** routing model on the validation set (10% of the dataset, 874 prompts), reporting the training loss, validation loss, and accuracy over the first 5 epochs of training. Note that the training setup specified 15 epochs (Section 2.3), but results are available only up to Epoch 5, possibly due to early stopping or partial reporting.

Table 1: Training and Validation Metrics for Routing Model (First 5 Epochs)

Epoch	Training Loss	Validation Loss	Accuracy
1	1.133000	1.101964	0.322222
2	1.123000	1.077411	0.333333
3	1.113000	1.057961	0.411111
4	1.113000	1.068906	0.322222
5	1.083900	1.046059	0.394444

The training loss decreases from 1.1330 to 1.0839 over 5 epochs, a 4.34% reduction, indicating that the model is gradually optimizing the cross-entropy loss on the training set. Meanwhile, the validation loss decreases more noticeably from 1.1019 to 1.0461 (a 5.07% drop), suggesting that the model is learning to generalize better on unseen data during the early stages of training. The accuracy improves from 0.3222 (32.22%) in epoch 1 to a peak of 0.4111 (41.11%) in epoch 3, before dropping again to 0.3222 (32.22%) in epoch 4 and slightly recovering to 0.3944 (39.44%) in epoch 5.

For a 3-class classification task, random guessing yields an expected accuracy of 33.33% (approximately 291 out of 873 samples). Thus, the highest observed accuracy of 41.11% represents a modest 7.78% absolute improvement over random guessing. However, the fluctuations in accuracy suggest instability in the model’s learning process, possibly due to label noise, over-regularization, or a learning rate that prevents consistent convergence. Further analysis in Section 4 explores how these results compare to baseline models and investigates potential improvements in training dynamics.

3.2 Cost Efficiency Evaluation

To assess the cost efficiency of our routing system, we compare the total cost of processing a test set of 1,745 prompts (approximately 20% of the dataset; note that this is an updated figure from the previously reported 10% split of 874 prompts) using three strategies: (1) a baseline that always routes to Mixtral-8x7B, (2) our trained router, which selects the LLM indicated by

the `soft_label_target` (derived from the highest soft label probability; see Section 2.2), and (3) a random selection strategy that chooses an LLM randomly for each prompt.

We selected Mixtral-8x7B as the baseline because it has the largest parameter count among the three LLMs in our study. With its mixture-of-experts architecture, effectively comprising 56 billion parameters (8 layers of 7B parameters each), we assumed it would provide the best results, particularly for complex queries, due to its increased capacity. However, this comes at a higher computational cost (\$0.6 per million tokens) compared to Mistral-7B (\$0.2 per million tokens), making it a suitable default choice for evaluating cost savings.

The total costs and percentage savings relative to the baseline are calculated as follows:

$$\text{savings} = 100 \times \frac{\text{baseline_cost} - \text{strategy_cost}}{\text{baseline_cost}}$$

The actual results are as follows:

Table 2: Cost Efficiency Comparison (Actual Results)

Strategy	Total Cost (USD)	Savings (%)
Router (Our Approach)	0.2439	19.4%
Random Selection	0.2806	7.3%

The results show that our routing system achieves a 19.4% cost saving compared to the baseline of always using Mixtral-8x7B, by intelligently selecting cheaper LLMs (e.g., Mistral-7B at \$0.2 per million tokens) when appropriate, while maintaining response quality and speed as encoded in the composite scores. The random selection strategy saves less (7.3%), as it does not consider quality or speed, often selecting more expensive models like Zero-One Yi-34B unnecessarily. The router’s performance aligns with its validation accuracy of 41.11% (Section 3.1), meaning approximately 40% of prompts may be misrouted to a more expensive LLM, limiting the savings compared to an ideal router.

Our cost savings are lower than those reported by PolyRouter [2], which achieved a 30% cost reduction using its BERT-Router compared to a Zero-Router baseline (average performance of standalone LLMs). Several factors contribute to this difference. First, our baseline of always using Mixtral-8x7B, a high-cost model, should theoretically allow for higher savings when avoided, but our router’s limited accuracy (41.11%) means it does not always select the cheapest LLM. PolyRouter’s BERT-Router, with an implied higher accuracy (inferred from its 11% BERTSim improvement), likely routes more prompts to cheaper models (e.g., Fox-1.6B at \$0.20 per million tokens). Second, our composite score approach, while more holistic by integrating cost directly into the soft labels, may introduce noise (e.g., conflicting signals between cost and quality), leading to less optimal routing decisions compared to PolyRouter’s BERT similarity-only soft labels. Finally, our inclusion of larger, more expensive LLMs (Mixtral-8x7B at \$0.6 per million tokens, Zero-One Yi-34B at \$0.8 per million) increases the baseline cost, but also means misrouting to these models has a higher cost impact. Despite the lower savings, our system demonstrates practical cost efficiency, particularly for a system designed to handle complex queries with larger LLMs, and highlights the trade-offs encoded in our composite score approach. Further analysis of the router’s performance is discussed in Section 4.

4 Discussion

Our system effectively addresses the LLM routing challenge by combining quality, cost, and throughput metrics into a composite score to generate soft labels. This approach enables the training of a `roberta-large` model for routing queries to Mistral-7B, Mixtral-8x7B, or Zero-One Yi-34B depending on their complexity. While our model achieves a cost saving of 19.4%

compared to a naive baseline of always using Mixtral-8x7B (Section 3.2), it also balances BERT similarity and throughput in routing decisions.

The routing model achieves an accuracy of 41.11% on a 3-class classification task, surpassing random guessing (33.33%) by 26.58%. Training loss decreased from 0.8310 to 0.8085 (2.7% reduction), while validation loss increased slightly from 0.8247 to 0.8267 (0.24% rise) across the first 5 epochs (Section 3.1). The lack of improvement in accuracy over epochs suggests convergence to a suboptimal solution, possibly due to a low learning rate ($2e-5$), overly soft labels (temperature $T = 0.5$), or overlapping prompt distributions across LLMs.

Compared to PolyRouter [2], which achieved a 30% cost reduction and 11% BERT similarity improvement using BERTSim-only soft labels, our method integrates throughput and cost into the label generation process. Despite our use of a larger model (**roberta-large** with 355M parameters versus **bert-base-uncased**'s 110M), our model underperforms in accuracy progression, potentially due to dataset limitations. The RouterBench dataset used (8,734 prompts) lacks the volume to fully exploit our model's capacity.

Several challenges impacted our deployment (Section 5). First, the Together API does not support Zero-One Yi-34B, necessitating a runtime substitution that introduces discrepancies in evaluation and cost assumptions (Section 2.4). Second, our composite score used equal weights for quality, cost, and speed ($w_1 = w_2 = w_3 = 1.0$), which may not suit all scenarios. Future work can explore dynamic weighting or user-configurable trade-offs. Additionally, temperature tuning in soft label generation ($T = 2.0$) could mitigate label softness.

The cost-saving evaluation assumes always using Mixtral-8x7B as the baseline. However, Mistral-7B frequently outperforms Mixtral-8x7B across multiple metrics, indicating that our baseline may overstate naive routing costs. PolyRouter's Zero-Router baseline, which averages over all models, may offer a fairer comparison. Alternative baselines such as always using Mistral-7B could be considered for broader evaluation.

The flat accuracy curve and moderate cost savings indicate room for improvement. Future efforts could include increasing the learning rate, extending training beyond 5 epochs, or simplifying to BERTSim-only soft labels. Despite the challenges, our system demonstrates the feasibility of using composite metrics for LLM routing and contributes a scalable, cost-efficient method with real-world deployment potential.

5 Challenges

Developing and deploying the multi-LLM routing system presented several challenges, which influenced the methodology and outcomes of the project. These difficulties highlight the practical complexities of building such a system and provide insights for future improvements.

First, dataset availability posed a significant challenge. Initially, we used the Nectar dataset, a large dataset primarily consisting of general chat conversations. However, the general nature of the chats led to poor routing performance, as they lacked the complexity and domain-specificity needed to effectively differentiate between LLMs tailored for easy, moderate, and complex queries. Consequently, we switched to the RouterBench dataset [1], which, while smaller with 8,734 prompts after filtering, covers diverse domains such as commonsense reasoning, math, coding, and conversation. This diversity better aligned with our routing objectives, enabling the model to learn domain-specific routing patterns, as evidenced by the sample metrics in Section 3. However, the smaller dataset size may have limited the model's ability to generalize, potentially contributing to the flat accuracy of 41.11% observed during training (Section 3.1).

Second, finding LLMs with freely available tokens for inference was challenging. While models like Mistral-7B and Mixtral-8x7B were accessible via the Together API with known pricing, accessing other LLMs, such as Zero-One Yi-34B, proved difficult due to API limitations. As noted in Section 2.4, the Together API did not support Zero-One Yi-34B, necessitating a substitution with another model. This limitation underscores the difficulty of sourcing LLMs

with compatible APIs and affordable token-based pricing, which can restrict the choice of models and impact deployment reliability.

Third, the initial dataset exhibited a bias toward one LLM, with most prompts being best handled by a single model, likely due to an imbalance in prompt complexity or domain suitability. This bias risked skewing the router’s training, causing it to over-predict the dominant LLM and neglect others, which would reduce cost savings and generalization. To address this, we filtered the dataset to ensure an equal number of prompts for each target LLM—Mistral-7B, Mixtral-8x7B, and Zero-One Yi-34B—resulting in a balanced dataset of approximately 2,911 prompts per LLM. This step, detailed in Section 2.3, was crucial for fair training and evaluation, contributing to the router’s ability to achieve a 41.11% accuracy, better than random guessing (33.33%). However, the filtering process may have further reduced the dataset’s diversity, potentially limiting the model’s learning capacity.

These challenges highlight the trade-offs involved in building a multi-LLM routing system, from dataset selection to model accessibility and data balancing. Addressing these issues through careful dataset curation, API exploration, and bias mitigation enabled the development of a functional routing system, but also points to areas for future improvement, as discussed in Section 6.

6 Conclusion

We present a multi-LLM routing system designed to direct queries to one of three LLMs—Mistral-7B, Mixtral-8x7B, and Zero-One Yi-34B—optimizing for response quality, computational cost, and inference speed. By computing BERT similarity scores, costs, and throughput, and deriving composite scores to generate soft labels, we trained a **roberta-large** model to predict the best LLM for a given prompt. Evaluated on a modified RouterBench dataset with 8,734 prompts, our system achieves a 19.4% cost saving compared to a baseline of always using Mixtral-8x7B, demonstrating its potential to balance the routing trilemma.

Our work extends the foundation laid by PolyRouter [2], introducing a novel composite score approach for soft label generation that integrates BERT similarity, cost, and throughput, surpassing PolyRouter’s BERT similarity-only method. This leads to competitive cost savings (19.4% vs. 30%) despite a lower router accuracy (41.11%), and enables more balanced routing decisions. Additionally, our focus on larger LLMs (up to 56 billion effective parameters) allows the system to handle more complex queries, and our use of RouterBench aligns with routing-specific benchmarks. However, limitations such as the Together API’s lack of support for Zero-One Yi-34B, requiring substitution with another model, highlight areas for improvement. This substitution introduces a mismatch between training and deployment, as the substitute model’s characteristics may differ from Zero-One Yi-34B.

Future work includes evaluating alternative APIs (e.g., xAI’s API) to support Zero-One Yi-34B, experimenting with composite score weights and temperature values to fine-tune routing decisions, and incorporating additional metrics like negative log-likelihood (as used by PolyRouter) to assess probabilistic quality. Inspired by PolyRouter’s edge-to-cloud vision, we also plan to explore deploying our router on edge devices, enabling local query processing with small LLMs while routing complex queries to the cloud. Our system provides a robust framework for multi-LLM routing, with clear contributions and a path for further enhancement.

References

- [1] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. *RouterBench: A Benchmark for Multi-LLM Routing System*. arXiv preprint arXiv:2403.12031, 2024.

- [2] Dimitris Stripelis, Zijian Hu, Jipeng Zhang, Zhaozhao Xu, Alay Dilipbhai Shah, Han Jin, Yuhang Yao, Salman Avestimehr, and Chaoyang He. *PolyRouter: A Multi-LLM Querying System*. arXiv preprint arXiv:2408.15533, 2024.